PERSONAL REQUIREMENTS FOR THE THIRD SEMESTER ASSIGNMENT:

- ***NAME*** : ***CHIMEZIE VICTOR OKONKWO***
- ***EMAIL ADDRESS : chimeziegodwins842@gmail.com***
- ***STUDENT ID : ALT/SOD/025/0137***

THE ASSIGNMENT : DATA ANALYSIS PRACTICAL

In [1]:
```
import pandas as pd

sales = pd.read_csv("C:/Users/USER/Documents/dirty_sales_dataset_with_georegion.csv

## We assigned our dataset to a variable so that we can reference it throughout thi
```

In [2]:
```
sales
```

Out[2]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue	GeoRe
0	999	2025-03-18	NaN	Abuja	5.0	-10000.0	-50000	
1	1000	2025-01-23	Laptop	NaN	6.0	339778.0	2038668	
2	1001	NaN	NaN	Lagos	-3.0	NaN	0	
3	1003	2025-03-09	Tablet	NaN	-3.0	NaN	unknown	
4	1003	invalid_date	NaN	Ibadan	-3.0	NaN	0	SC
...	...	...	...	...	...	...	...	
95	1095	2025-04-09	PHONE	Lagos	-3.0	-10000.0	30000	SC
96	1096	invalid_date	NaN	Abuja	-3.0	NaN	NaN	
97	1097	NaN	Phone	NaN	4.0	222901.0	891604	SC
98	1097	NaN	PHONE	NaN	NaN	-10000.0	unknown	
99	1098	invalid_date	laptop	Lagos	-3.0	NaN	unknown	

100 rows × 8 columns

- ***SECTION A : CLEANING IDENTIFICATION : IDENTIFYING MISSING VALUES, WRONG TYPES, OUTLIERS, AND FORMATTING ISSUES BEFORE FIXING THEM.***
- ***DATA EXPLORATION***

- ***Firstly, before analysis, we used the .info() function to determine the content of our dataset. We do this so as to ;***
- ***-Know how and where to identify missing values.***
- ***-Identify outliers and wrong types/misplaced values.***
- ***-Identify any issues that our dataset might contain.***

In [3]:
```
sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   OrderID       100 non-null    int64  
 1   Date          76 non-null     object 
 2   Product       85 non-null     object 
 3   Region        67 non-null     object 
 4   UnitsSold     73 non-null     float64
 5   UnitPrice     63 non-null     float64
 6   TotalRevenue  77 non-null     object 
 7   GeoRegion     86 non-null     object 
dtypes: float64(2), int64(1), object(5)
memory usage: 6.4+ KB
```

- ***Another method of running a quick summary statistics check is to use the .describe() function.***
- ***Basically, this function runs a quick summary of some of the basic aggregate functions on all numerical values in our dataset.***

In [4]:
```
sales.describe()
```

Out[4]:

	OrderID	UnitsSold	UnitPrice
count	100.000000	73.000000	63.000000
mean	1048.960000	1.246575	151664.063492
std	29.115902	4.639147	169748.958463
min	999.000000	-3.000000	-10000.000000
25%	1024.000000	-3.000000	-10000.000000
50%	1049.000000	1.000000	133036.000000
75%	1074.000000	5.000000	314933.500000
max	1098.000000	10.000000	488492.000000

- ***When exploring the dataset, we might need to count the frequency of unique values within the dataset.***
- ***This is when we use the .value_counts() function to explore the dataset.***

In [5]:
```
sales.value_counts()
```

Out[5]:
```
OrderID  Date          Product     Region  UnitsSold  UnitPrice  TotalRevenue  Geo
Region
1026     15/03/2025    laptop      lagos    5.0        376839.0  1884195       nor
th        1
1028     2025-04-15    Headphones  Ibadan  -3.0       -10000.0   30000         Nor
th        1
1037     2025-04-16    Laptop      Lagos    7.0        144375.0  1010625       Sou
th        1
1042     invalid_date  Tablet      Ibadan  -3.0       -10000.0   30000         Nor
th        1
1043     2025-01-03    laptop      Lagos    1.0        77264.0   77264         SOU
TH        1
1050     2025-04-07    Headphones  Abuja    8.0       -10000.0   -80000        nor
th        1
1051     invalid_date  PHONE       Lagos    5.0        317391.0  1586955       Nor
th        1
1054     2025-03-07    PHONE       Abuja    8.0        383170.0  3065360       Wes
t         1
1056     26/01/2025    Phone       Kano    -3.0        137095.0  -411285       EAS
T         1
1087     invalid_date  Laptop      Ibadan  -3.0       -10000.0   30000         Sou
th        1
1089     invalid_date  Tablet      lagos   -3.0       -10000.0   30000         SOU
TH        1
1092     24/02/2025    laptop      Kano     6.0        75679.0   454074        SOU
TH        1
         28/03/2025    Tablet      Lagos   -3.0        468110.0  -1404330      Wes
t         1
1094     invalid_date  laptop      Ibadan  -3.0        156686.0  -470058       Wes
t         1
1095     2025-04-09    PHONE       Lagos   -3.0       -10000.0   30000         SOU
TH        1
Name: count, dtype: int64
```

- ***We can narrow it down to determine the frequency of unique values in a particular column.***
- ***For example, when we use this function on the 'Region' column, we see how frequent each region made sales.***

In [6]:
```
sales['Region'].value_counts()
```

Out[6]:
```
Region
Abuja            15
Ibadan           14
Lagos            12
Kano             10
Port Harcourt     8
lagos             8
Name: count, dtype: int64
```

- ***The .unique() function almost does what the .value_counts does, but there's a tiny different detail.***
- ***This function calls out the regions but this time, it includes the null/empty/wrong_type values.***

In [7]:
```
sales['Region'].unique()
```

Out[7]:
```
array(['Abuja', nan, 'Lagos', 'Ibadan', 'Kano', 'Port Harcourt', 'lagos'],
      dtype=object)
```

- ***This exploratory method calls out the first five rows by default.***
- ***Also, it can take one parameter but it has to be set to the number of rows you want to see.***

In [8]:
```
sales.head(10)
```

Out[8]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue	GeoRe
0	999	2025-03-18	NaN	Abuja	5.0	-10000.0	-50000	
1	1000	2025-01-23	Laptop	NaN	6.0	339778.0	2038668	
2	1001	NaN	NaN	Lagos	-3.0	NaN	0	
3	1003	2025-03-09	Tablet	NaN	-3.0	NaN	unknown	
4	1003	invalid_date	NaN	Ibadan	-3.0	NaN	0	SC
5	1004	invalid_date	Laptop	Kano	NaN	NaN	NaN	
6	1006	24/04/2025	Tablet	NaN	NaN	-10000.0	0	r
7	1006	2025-04-29	Tablet	NaN	-3.0	NaN	NaN	
8	1007	NaN	NaN	NaN	NaN	199284.0	unknown	
9	1008	03/02/2025	NaN	Port Harcourt	5.0	NaN	unknown	

- ***This is the direct opposite of the .head() function. By default, it calls out the last five rows.***

- ***And also, just like the head function, you can set it to the number of rows you want to see.***

In [9]:
```
sales.tail(8)
```

Out[9]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue	GeoRe
92	1092	24/02/2025	laptop	Kano	6.0	75679.0	454074	SC
93	1092	28/03/2025	Tablet	Lagos	-3.0	468110.0	-1404330	
94	1094	invalid_date	laptop	Ibadan	-3.0	156686.0	-470058	
95	1095	2025-04-09	PHONE	Lagos	-3.0	-10000.0	30000	SC
96	1096	invalid_date	NaN	Abuja	-3.0	NaN	NaN	
97	1097	NaN	Phone	NaN	4.0	222901.0	891604	SC
98	1097	NaN	PHONE	NaN	NaN	-10000.0	unknown	
99	1098	invalid_date	laptop	Lagos	-3.0	NaN	unknown	

- ***The .isnull() function : This is used to call out null/NaN values in a dataset.***
- ***Just like Boolean masking, the values labeled 'True' are the null values because they met the condition, which makes them "true".***

In [10]:
```
sales.isnull()
```

Out[10]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue	GeoRegion
0	False	False	True	False	False	False	False	False
1	False	False	False	True	False	False	False	True
2	False	True	True	False	False	True	False	False
3	False	False	False	True	False	True	False	False
4	False	False	True	False	False	True	False	False
...	...	...	...	...	...	...	...	...
95	False	False	False	False	False	False	False	False
96	False	False	True	False	False	True	True	False
97	False	True	False	True	False	False	False	False
98	False	True	False	True	True	False	False	False
99	False	False	False	False	False	True	False	False

100 rows × 8 columns

- ***We could narrow this function down to give us only the null values in a particular column.***
- ***For example, we tested this function on the 'Product' column. It listed out only the rows that had null values in the 'Product' column.***

In [11]:
```
sales[sales['Product'].isnull()]
```

Out[11]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue	GeoR
0	999	2025-03-18	NaN	Abuja	5.0	-10000.0	-50000	
2	1001	NaN	NaN	Lagos	-3.0	NaN	0	
4	1003	invalid_date	NaN	Ibadan	-3.0	NaN	0	S
8	1007	NaN	NaN	NaN	NaN	199284.0	unknown	
9	1008	03/02/2025	NaN	Port Harcourt	5.0	NaN	unknown	
28	1028	NaN	NaN	lagos	-3.0	NaN	unknown	
31	1031	invalid_date	NaN	Ibadan	9.0	-10000.0	-90000	S
35	1034	invalid_date	NaN	lagos	NaN	NaN	NaN	
40	1039	NaN	NaN	NaN	NaN	-10000.0	unknown	
47	1046	13/01/2025	NaN	Ibadan	-3.0	NaN	0	S
51	1050	2025-02-28	NaN	NaN	2.0	NaN	NaN	
53	1052	NaN	NaN	Ibadan	NaN	147556.0	NaN	
55	1054	NaN	NaN	NaN	-3.0	NaN	unknown	
76	1075	invalid_date	NaN	Ibadan	-3.0	-10000.0	30000	
96	1096	invalid_date	NaN	Abuja	-3.0	NaN	NaN	

- ***SECTION B : DATA CLEANING : ACTUALLY FIXING THE IDENTIFIED ISSUES AND VERIFYING THE RESULTS WITH CODE.***
- ***DATA CLEANING***

- ***When we begin to clean data, it's very important that we work on the datatypes. This is because the optimal datatype allows us to perform different calculations/operations on the dataset. Also, this helps us fix issues such as the negative numerical figures/floats on the numerical columns.***

- ***In order to change datatypes, we explore different codes for various purposes. For example ;***

In [12]:
```
sales['OrderID'] = sales['OrderID'].astype('object')
sales['UnitsSold'] = sales['UnitsSold'].astype('float64')
sales['UnitPrice'] = sales['UnitPrice'].astype('Int64')
```

- ***When we check the info of out dataset, we've successfully changed some of the datatypes.***

In [13]:
```
sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   OrderID       100 non-null    object 
 1   Date          76 non-null     object 
 2   Product       85 non-null     object 
 3   Region        67 non-null     object 
 4   UnitsSold     73 non-null     float64
 5   UnitPrice     63 non-null     Int64  
 6   TotalRevenue  77 non-null     object 
 7   GeoRegion     86 non-null     object 
dtypes: Int64(1), float64(1), object(6)
memory usage: 6.5+ KB
```

- ***We'll use a different code to change other columns' datatypes.***

In [14]:
```
sales['Date'] = pd.to_datetime(sales['Date'], format = 'mixed', errors = 'coerce')
```

- ***I encountered errors when writing this above code;***
- ***1. I use the 'format' code to convert some of the dates and***
- ***2. I used the "errors = 'coerce'" to force the invalid dates into "NaT".***

In [15]:
```
sales
```

Out[15]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue	GeoRegion
0	999	2025-03-18	NaN	Abuja	5.0	-10000	-50000	??
1	1000	2025-01-23	Laptop	NaN	6.0	339778	2038668	NaN
2	1001	NaT	NaN	Lagos	-3.0	<NA>	0	EAST
3	1003	2025-03-09	Tablet	NaN	-3.0	<NA>	unknown	East
4	1003	NaT	NaN	Ibadan	-3.0	<NA>	0	SOUTH
...	...	...	...	...	...	...	...	...
95	1095	2025-04-09	PHONE	Lagos	-3.0	-10000	30000	SOUTH
96	1096	NaT	NaN	Abuja	-3.0	<NA>	NaN	East
97	1097	NaT	Phone	NaN	4.0	222901	891604	SOUTH
98	1097	NaT	PHONE	NaN	NaN	-10000	unknown	EAST
99	1098	NaT	laptop	Lagos	-3.0	<NA>	unknown	west

100 rows × 8 columns

- ***Now, when we view the info of our dataset, the 'Date' column have been successfully transformed.***

In [16]:
```
sales.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype         
---  ------        --------------  -----         
 0   OrderID       100 non-null    object        
 1   Date          45 non-null     datetime64[ns]
 2   Product       85 non-null     object        
 3   Region        67 non-null     object        
 4   UnitsSold     73 non-null     float64       
 5   UnitPrice     63 non-null     Int64         
 6   TotalRevenue  77 non-null     object        
 7   GeoRegion     86 non-null     object        
dtypes: Int64(1), datetime64[ns](1), float64(1), object(5)
memory usage: 6.5+ KB
```

- ***Also, to make the 'TotalRevenue' column to be all positive figures, we'll use other different codes.***

In [17]:
```
sales['TotalRevenue'] = pd.to_numeric(sales['TotalRevenue'], errors = 'coerce')
sales['TotalRevenue'] = sales['TotalRevenue'].abs()
```

- ***In this code above, I had to fix 2 issues in the 'TotalRevenue' column ;***
- ***1. I had to change all the rows into numeric values and then, force the strings in that column into 'NaN'.***
- ***2. I had to convert the negative numbers into positive numbers.***

In [18]:
```
sales
```

Out[18]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue	GeoRegion
0	999	2025-03-18	NaN	Abuja	5.0	-10000	50000.0	??
1	1000	2025-01-23	Laptop	NaN	6.0	339778	2038668.0	NaN
2	1001	NaT	NaN	Lagos	-3.0	<NA>	0.0	EAST
3	1003	2025-03-09	Tablet	NaN	-3.0	<NA>	NaN	East
4	1003	NaT	NaN	Ibadan	-3.0	<NA>	0.0	SOUTH
...	...	...	...	...	...	...	...	...
95	1095	2025-04-09	PHONE	Lagos	-3.0	-10000	30000.0	SOUTH
96	1096	NaT	NaN	Abuja	-3.0	<NA>	NaN	East
97	1097	NaT	Phone	NaN	4.0	222901	891604.0	SOUTH
98	1097	NaT	PHONE	NaN	NaN	-10000	NaN	EAST
99	1098	NaT	laptop	Lagos	-3.0	<NA>	NaN	west

100 rows × 8 columns

- ***USING THE .abs() METHOD, WE WILL CHANGE ALL THE NEGATIVE NUMBERS IN THE 'UnitPrice' AND 'UnitsSold' COLUMNS INTO POSITIVE NUMBERS.***

In [19]:
```
sales['UnitPrice'] = sales['UnitPrice'].abs()
sales['UnitsSold'] = sales['UnitsSold'].abs()
```

In [20]:
```
sales
```

Out[20]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue	GeoRegion
0	999	2025-03-18	NaN	Abuja	5.0	10000	50000.0	??
1	1000	2025-01-23	Laptop	NaN	6.0	339778	2038668.0	NaN
2	1001	NaT	NaN	Lagos	3.0	<NA>	0.0	EAST
3	1003	2025-03-09	Tablet	NaN	3.0	<NA>	NaN	East
4	1003	NaT	NaN	Ibadan	3.0	<NA>	0.0	SOUTH
...	...	...	...	...	...	...	...	...
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	30000.0	SOUTH
96	1096	NaT	NaN	Abuja	3.0	<NA>	NaN	East
97	1097	NaT	Phone	NaN	4.0	222901	891604.0	SOUTH
98	1097	NaT	PHONE	NaN	NaN	10000	NaN	EAST
99	1098	NaT	laptop	Lagos	3.0	<NA>	NaN	west

100 rows × 8 columns

- ***I CREATED A COPY OF THE DATASET SO THAT I CAN KEEP THE ORIGINAL FOR REFERENCE PURPOSES.***
- ***TO DO THIS, THERE'S A FUNCTION TO BE USED; THE COPY FUNCTION.***

In [21]:
```
sales_copy = sales.copy()
```

In [22]:
```
sales_copy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype         
---  ------        --------------  -----         
 0   OrderID       100 non-null    object        
 1   Date          45 non-null     datetime64[ns]
 2   Product       85 non-null     object        
 3   Region        67 non-null     object        
 4   UnitsSold     73 non-null     float64       
 5   UnitPrice     63 non-null     Int64         
 6   TotalRevenue  57 non-null     float64       
 7   GeoRegion     86 non-null     object        
dtypes: Int64(1), datetime64[ns](1), float64(2), object(4)
memory usage: 6.5+ KB
```

- ***HENCEFORTH, WE'LL USE THE COPY OF THE DATASET TO CONTINUE THE DATA CLEANING PROCESS.***

In [23]:
```
sales_copy
```

Out[23]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue	GeoRegion
0	999	2025-03-18	NaN	Abuja	5.0	10000	50000.0	??
1	1000	2025-01-23	Laptop	NaN	6.0	339778	2038668.0	NaN
2	1001	NaT	NaN	Lagos	3.0	<NA>	0.0	EAST
3	1003	2025-03-09	Tablet	NaN	3.0	<NA>	NaN	East
4	1003	NaT	NaN	Ibadan	3.0	<NA>	0.0	SOUTH
...	...	...	...	...	...	...	...	...
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	30000.0	SOUTH
96	1096	NaT	NaN	Abuja	3.0	<NA>	NaN	East
97	1097	NaT	Phone	NaN	4.0	222901	891604.0	SOUTH
98	1097	NaT	PHONE	NaN	NaN	10000	NaN	EAST
99	1098	NaT	laptop	Lagos	3.0	<NA>	NaN	west

100 rows × 8 columns

- ***IF WE WANT TO REMOVE VALUES IN OUR DATASET, LET'S SAY DATA WE WON'T NEED. WE USE THE 'DROP' FUNCTION.***
- ***FOR EXAMPLE; LET'S REMOVE THE 'GeoRegion' COLUMN.***

In [24]:
```
sales_copy.drop('GeoRegion', axis = 1)
```

Out[24]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue
0	999	2025-03-18	NaN	Abuja	5.0	10000	50000.0
1	1000	2025-01-23	Laptop	NaN	6.0	339778	2038668.0
2	1001	NaT	NaN	Lagos	3.0	<NA>	0.0
3	1003	2025-03-09	Tablet	NaN	3.0	<NA>	NaN
4	1003	NaT	NaN	Ibadan	3.0	<NA>	0.0
...	...	...	...	...	...	...	...
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	30000.0
96	1096	NaT	NaN	Abuja	3.0	<NA>	NaN
97	1097	NaT	Phone	NaN	4.0	222901	891604.0
98	1097	NaT	PHONE	NaN	NaN	10000	NaN
99	1098	NaT	laptop	Lagos	3.0	<NA>	NaN

100 rows × 7 columns

- ***This function above takes two parameters; the column name and the axis position of that column (rows are axis 0, columns are axis 1).***

- ***For example; if we drop 'TotalRevenue' and want to make it permanent, we use the 'inplace' function.***

In [25]:
```
sales_copy.drop('TotalRevenue', axis = 1, inplace = True)
```

- ***When we call our dataset again, we can see that the change is now permanent.***

In [26]:
```
sales_copy
```

Out[26]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	GeoRegion
0	999	2025-03-18	NaN	Abuja	5.0	10000	??
1	1000	2025-01-23	Laptop	NaN	6.0	339778	NaN
2	1001	NaT	NaN	Lagos	3.0	<NA>	EAST
3	1003	2025-03-09	Tablet	NaN	3.0	<NA>	East
4	1003	NaT	NaN	Ibadan	3.0	<NA>	SOUTH
...	...	...	...	...	...	...	...
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	SOUTH
96	1096	NaT	NaN	Abuja	3.0	<NA>	East
97	1097	NaT	Phone	NaN	4.0	222901	SOUTH
98	1097	NaT	PHONE	NaN	NaN	10000	EAST
99	1098	NaT	laptop	Lagos	3.0	<NA>	west

100 rows × 7 columns

- ***THE 'DROPNA' FUNCTION :***
- ***This is another type of the 'drop' function. This one is more extreme. What it does; it eliminates rows or columns with any missing or null values, entirely. By default, it eliminates rows unless when specified in it's parameters.***

In [27]:
```
sales_copy.dropna()
```

Out[27]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	GeoRegion
26	1026	2025-03-15	laptop	lagos	5.0	376839	north
29	1028	2025-04-15	Headphones	Ibadan	3.0	10000	North
37	1037	2025-04-16	Laptop	Lagos	7.0	144375	South
44	1043	2025-01-03	laptop	Lagos	1.0	77264	SOUTH
50	1050	2025-04-07	Headphones	Abuja	8.0	10000	north
54	1054	2025-03-07	PHONE	Abuja	8.0	383170	West
56	1056	2025-01-26	Phone	Kano	3.0	137095	EAST
92	1092	2025-02-24	laptop	Kano	6.0	75679	SOUTH
93	1092	2025-03-28	Tablet	Lagos	3.0	468110	West
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	SOUTH

- ***The code above returned very few values because most of the dataset have missing or null values in the rows.***

- ***To show another example of the above function on the columns side, we'll introduce another function called the "Threshold" function.***
- ***THRESHOLD FUNCTION :***
- ***The Threshold function refers to the number of non-null values each column/row should have. If a column/row doesn't have up to the number stated in the threshold function, it gets dropped.***

In [28]:
```
sales_copy.dropna(axis=1, thresh=30)
```

Out[28]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	GeoRegion
0	999	2025-03-18	NaN	Abuja	5.0	10000	??
1	1000	2025-01-23	Laptop	NaN	6.0	339778	NaN
2	1001	NaT	NaN	Lagos	3.0	<NA>	EAST
3	1003	2025-03-09	Tablet	NaN	3.0	<NA>	East
4	1003	NaT	NaN	Ibadan	3.0	<NA>	SOUTH
...	...	...	...	...	...	...	...
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	SOUTH
96	1096	NaT	NaN	Abuja	3.0	<NA>	East
97	1097	NaT	Phone	NaN	4.0	222901	SOUTH
98	1097	NaT	PHONE	NaN	NaN	10000	EAST
99	1098	NaT	laptop	Lagos	3.0	<NA>	west

100 rows × 7 columns

- ***In the code above:***
- ***1. The 'Date' column got dropped because it didn't have up to 30 non-null values.***
- ***2. I used the axis 1 (the columns axis) to explain this function.***

- ***USING SUBSETS IN THE DROP FUNCTION***
- ***What I mean, to use subset in the 'drop' function is; we might want to drop rows/columns specifically because of the null values in that particular row or column.***

In [29]:
```
sales_copy.dropna(axis=0, subset = ['Product'])

## Now, I eliminated all the null values in the 'Product' column.
## I used the rows axis (axis 0) to explain this code.
```

Out[29]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	GeoRegion
1	1000	2025-01-23	Laptop	NaN	6.0	339778	NaN
3	1003	2025-03-09	Tablet	NaN	3.0	<NA>	East
5	1004	NaT	Laptop	Kano	NaN	<NA>	East
6	1006	2025-04-24	Tablet	NaN	NaN	10000	north
7	1006	2025-04-29	Tablet	NaN	3.0	<NA>	NaN
...	...	...	...	...	...	...	...
94	1094	NaT	laptop	Ibadan	3.0	156686	West
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	SOUTH
97	1097	NaT	Phone	NaN	4.0	222901	SOUTH
98	1097	NaT	PHONE	NaN	NaN	10000	EAST
99	1098	NaT	laptop	Lagos	3.0	<NA>	west

85 rows × 7 columns

- ***FILLNA FUNCTION :***
- ***Another soft version of the 'dropna' function is the 'fillna' function. Now, this function allows you to fill in the null values in the dataset with anything of your choice.***

In [30]:
```
## For example;

import pandas as pd

sales_copy['UnitsSold'] = sales_copy['UnitsSold'].fillna(0)
sales_copy['UnitPrice'] = sales_copy['UnitPrice'].fillna(1)
sales_copy['Product'] = sales_copy['Product'].fillna('Unspecified')
sales_copy['Region'] = sales_copy['Region'].fillna('Unidentified')
sales_copy['Date'] = sales_copy['Date'].fillna(pd.to_datetime('1900-01-01'))
sales_copy['GeoRegion'] = sales_copy['GeoRegion'].fillna('Unspecified')
```

```
sales_copy
```

Out[30]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	GeoRegion
0	999	2025-03-18	Unspecified	Abuja	5.0	10000	??
1	1000	2025-01-23	Laptop	Unidentified	6.0	339778	Unspecified
2	1001	1900-01-01	Unspecified	Lagos	3.0	1	EAST
3	1003	2025-03-09	Tablet	Unidentified	3.0	1	East
4	1003	1900-01-01	Unspecified	Ibadan	3.0	1	SOUTH
...	...	...	...	...	...	...	...
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	SOUTH
96	1096	1900-01-01	Unspecified	Abuja	3.0	1	East
97	1097	1900-01-01	Phone	Unidentified	4.0	222901	SOUTH
98	1097	1900-01-01	PHONE	Unidentified	0.0	10000	EAST
99	1098	1900-01-01	laptop	Lagos	3.0	1	west

100 rows × 7 columns

- ***When I check the dataset's info, there's no null values in the dataset now. All of them have been filled up.***

In [31]:
```
sales_copy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   OrderID    100 non-null    object
 1   Date       100 non-null    datetime64[ns]
 2   Product    100 non-null    object
 3   Region     100 non-null    object
 4   UnitsSold  100 non-null    float64
 5   UnitPrice  100 non-null    Int64
 6   GeoRegion  100 non-null    object
dtypes: Int64(1), datetime64[ns](1), float64(1), object(4)
memory usage: 5.7+ KB
```

- ***SECTION C : EDA CLEANING TOWARD ANALYTICS***

- ***CHECKING FOR DUPLICATES***
- ***We check for duplicates in our dataset by using the .duplicated() function.***
- ***To make sure there are no duplicated values, we add .value_counts() to see the total number of values.***

In [32]:
```
sales_copy.duplicated().value_counts()
```

Out[32]:
```
False    100
Name: count, dtype: int64
```

- ***We could also check for duplicates on a specific column or multiple columns combined by using subset.***

In [33]:
```
sales_copy.duplicated(subset = ['UnitPrice'])
```

Out[33]:
```
0     False
1     False
2     False
3      True
4      True
      ...
95     True
96     True
97    False
98     True
99     True
Length: 100, dtype: bool
```

- ***The duplicates it found are the null values or the ones with 0 as their value.***

- ***KEEPING DUPLICATES***
- ***We can also keep certain duplicates that we want.***

- ***In this code below, we're telling Python to keep the first of each duplicate and then flag the rest as duplicates.***

In [34]:
```
sales_copy.duplicated(subset = ['Region'], keep = 'first')
```

Out[34]:
```
0     False
1     False
2     False
3      True
4     False
      ...
95     True
96     True
97     True
98     True
99     True
Length: 100, dtype: bool
```

- ***If we want to see the duplicates, we write this code below;***

In [35]:
```
sales[sales.duplicated(keep = False)]
```

Out[35]:

OrderID	Date	Product	Region	UnitsSold	UnitPrice	TotalRevenue	GeoRegion

- ***So, from the above code, we can see there are no longer duplicates in the dataset.***

- ***ADDING NEW COLUMNS TO THE DATAFRAME.***
- ***We can add new columns to our dataset when we want to by using different methods to do so. Here, we'll create a new column by using arithmetic functions to derive it.***

In [36]:
```
sales_copy['Total_Amount'] = (sales_copy['UnitsSold'] * sales_copy['UnitPrice'])

sales_copy
```

Out[36]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	GeoRegion	Total_A
0	999	2025-03-18	Unspecified	Abuja	5.0	10000	??	5
1	1000	2025-01-23	Laptop	Unidentified	6.0	339778	Unspecified	203
2	1001	1900-01-01	Unspecified	Lagos	3.0	1	EAST	
3	1003	2025-03-09	Tablet	Unidentified	3.0	1	East	
4	1003	1900-01-01	Unspecified	Ibadan	3.0	1	SOUTH	
...	...	...	...	...	...	...	...	
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	SOUTH	3
96	1096	1900-01-01	Unspecified	Abuja	3.0	1	East	
97	1097	1900-01-01	Phone	Unidentified	4.0	222901	SOUTH	89
98	1097	1900-01-01	PHONE	Unidentified	0.0	10000	EAST	
99	1098	1900-01-01	laptop	Lagos	3.0	1	west	

100 rows × 8 columns

In [37]:
```
sales_copy['Discount(Percentage)'] = (sales_copy['UnitsSold'] * sales_copy['UnitPri

sales_copy
```

Out[37]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	GeoRegion	Total_A
0	999	2025-03-18	Unspecified	Abuja	5.0	10000	??	5
1	1000	2025-01-23	Laptop	Unidentified	6.0	339778	Unspecified	203
2	1001	1900-01-01	Unspecified	Lagos	3.0	1	EAST	
3	1003	2025-03-09	Tablet	Unidentified	3.0	1	East	
4	1003	1900-01-01	Unspecified	Ibadan	3.0	1	SOUTH	
...	...	...	...	...	...	...	...	
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	SOUTH	3
96	1096	1900-01-01	Unspecified	Abuja	3.0	1	East	
97	1097	1900-01-01	Phone	Unidentified	4.0	222901	SOUTH	89
98	1097	1900-01-01	PHONE	Unidentified	0.0	10000	EAST	
99	1098	1900-01-01	laptop	Lagos	3.0	1	west	

100 rows × 9 columns

- ***We could also add another column by importing numpy and using its functions.***

In [38]:
```
import numpy as np

sales_copy['Ratings'] = np.random.randint(0, 10, size = len(sales_copy))
```

In [39]:
```
sales_copy
```

Out[39]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	GeoRegion	Total_A
0	999	2025-03-18	Unspecified	Abuja	5.0	10000	??	5
1	1000	2025-01-23	Laptop	Unidentified	6.0	339778	Unspecified	203
2	1001	1900-01-01	Unspecified	Lagos	3.0	1	EAST	
3	1003	2025-03-09	Tablet	Unidentified	3.0	1	East	
4	1003	1900-01-01	Unspecified	Ibadan	3.0	1	SOUTH	
...	...	...	...	...	...	...	...	
95	1095	2025-04-09	PHONE	Lagos	3.0	10000	SOUTH	3
96	1096	1900-01-01	Unspecified	Abuja	3.0	1	East	
97	1097	1900-01-01	Phone	Unidentified	4.0	222901	SOUTH	89
98	1097	1900-01-01	PHONE	Unidentified	0.0	10000	EAST	
99	1098	1900-01-01	laptop	Lagos	3.0	1	west	

100 rows × 10 columns

- ***THE "LAMBDA FUNCTION" :***
- ***The Lambda function is an anonymous function that can be defined without a name/variable. Oftentimes, it's written as a single line of code mostly used for short or quick tasks.***
- ***In the code below, we'll use a lambda function to make all rows in a particular column to be in uppercase letters.***

In [40]:
```
sales_copy['GeoRegion'] = sales_copy['GeoRegion'].apply(lambda x: x.upper())
sales_copy['Product'] = sales_copy['Product'].apply(lambda x: x.upper())
sales_copy['Region'] = sales_copy['Region'].apply(lambda x: x.upper())

sales_copy
```

Out[40]:

	OrderID	Date	Product	Region	UnitsSold	UnitPrice	GeoRegion	To
0	999	2025-03-18	UNSPECIFIED	ABUJA	5.0	10000	??	
1	1000	2025-01-23	LAPTOP	UNIDENTIFIED	6.0	339778	UNSPECIFIED	
2	1001	1900-01-01	UNSPECIFIED	LAGOS	3.0	1	EAST	
3	1003	2025-03-09	TABLET	UNIDENTIFIED	3.0	1	EAST	
4	1003	1900-01-01	UNSPECIFIED	IBADAN	3.0	1	SOUTH	
...	...	...	...	...	...	...	...	
95	1095	2025-04-09	PHONE	LAGOS	3.0	10000	SOUTH	
96	1096	1900-01-01	UNSPECIFIED	ABUJA	3.0	1	EAST	
97	1097	1900-01-01	PHONE	UNIDENTIFIED	4.0	222901	SOUTH	
98	1097	1900-01-01	PHONE	UNIDENTIFIED	0.0	10000	EAST	
99	1098	1900-01-01	LAPTOP	LAGOS	3.0	1	WEST	

100 rows × 10 columns

- ***SECTION D : DATA ANALYTICS USING AGGREGATE FUNCTIONS***
- ***EDA CLEANING : INVESTIGATIVE AND EXPLORATORY ANALYTICS.***

- ***GROUP BY***
- ***Just like in SQL, we can use the .groupby() function to aggregate the values of the numeric columns in the dataset.***
- ***Below is an example of a sum groupby of 'Product'.***

In [41]:
```
Product_groupby = sales_copy.groupby('Product').sum(numeric_only=True)

Product_groupby
```

Out[41]:

Product	UnitsSold	UnitPrice	Total_Amount	Discount(Percentage)	Ratings
HEADPHONES	46.0	619016	3264782.0	32647.82	67
LAPTOP	94.0	3704382	16127823.0	161278.23	75
MONITOR	8.0	806768	1465481.0	14654.81	22
PHONE	82.0	3916706	14277410.0	142774.1	129
TABLET	35.0	641152	2292563.0	22925.63	47
UNSPECIFIED	42.0	386849	170025.0	1700.25	72

- ***IF WE DECIDE TO SEE THE TOTAL AMOUNT OF THESE PRODUCTS INDIVIDUALLY, WE DO THIS BELOW ;***

In [42]:
```
Product_groupby1 = sales_copy.groupby('Product').sum(numeric_only=True)['Total_Amou

Product_groupby1
```

Out[42]:
```
Product
HEADPHONES      3264782.0
LAPTOP         16127823.0
MONITOR         1465481.0
PHONE          14277410.0
TABLET          2292563.0
UNSPECIFIED      170025.0
Name: Total_Amount, dtype: Float64
```

- ***WE COULD ALSO COMBINE THESE AGGREGATE FUNCTION IN A WAY THAT WE CAN SEE MULTIPLE AGGREGATIONS IN A SINGLE DATAFRAME.***

In [43]:
```
Product_groupby2 = sales_copy.agg({'Total_Amount':'sum', 'Ratings':'mean'})

Product_groupby2
```

Out[43]:
```
Total_Amount    37598084.00
Ratings                4.12
dtype: float64
```

- ***MULTI INDEX DATAFRAME :***
- ***THIS IS WHEN WE CREATE A BIGGER AND MORE COMPLEX DATAFRAME OF THE DATASET TO OBSERVE MORE DETAILED INFORMATION OF THE DATASET.***
- ***BELOW IS A GOOD EXAMPLE OF A MULTI INDEX DATAFRAME.***

In [44]:
```
reg_prod_agg = sales_copy.groupby(['Region', 'Product'])
reg_prod_agg.mean(numeric_only = True)
```

Out[44]:

Region	Product	UnitsSold	UnitPrice	Total_Amount	Discount(Perc
ABUJA	HEADPHONES	4.750000	2500.75	20002.75	2
	LAPTOP	6.500000	1.0	6.5	
	PHONE	2.666667	277977.0	1021786.666667	10217
	TABLET	2.250000	2500.75	7501.5	
	UNSPECIFIED	4.000000	5000.5	25001.5	
IBADAN	HEADPHONES	3.000000	10000.0	30000.0	
	LAPTOP	3.000000	160148.0	480444.0	
	MONITOR	3.000000	488492.0	1465476.0	1
	PHONE	1.500000	91856.5	91857.0	
	TABLET	3.000000	10000.0	30000.0	
	UNSPECIFIED	3.600000	33511.6	24001.2	
KANO	HEADPHONES	0.000000	85511.333333	0.0	
	LAPTOP	2.000000	140049.2	197859.6	1
	PHONE	1.500000	73547.5	205642.5	2
LAGOS	HEADPHONES	2.000000	1.0	2.0	
	LAPTOP	5.000000	119696.0	594419.2	5
	MONITOR	2.500000	5000.5	2.5	
	PHONE	3.000000	107952.142857	390177.142857	390
	TABLET	3.000000	239055.0	717165.0	
	UNSPECIFIED	2.000000	1.0	2.0	
PORT HARCOURT	HEADPHONES	0.000000	10000.0	0.0	
	MONITOR	0.000000	308274.0	0.0	
	PHONE	2.600000	69009.0	6002.0	
	UNSPECIFIED	5.000000	1.0	5.0	
UNIDENTIFIED	HEADPHONES	4.400000	66495.6	630953.8	6
	LAPTOP	4.250000	220632.75	1280580.0	
	MONITOR	0.000000	1.0	0.0	
	PHONE	3.400000	165125.7	785580.1	7

Region	Product	UnitsSold	UnitPrice	Total_Amount	Discount(Perc
	TABLET	3.400000	28607.8	159645.4	
	UNSPECIFIED	1.250000	52321.5	1.25	

- ***FOR EXAMPLE; IF WE WANT TO SEE SALES SPECIFIC TO A REGION, e.g 'Abuja', WE DO THE FOLLOWING BELOW.***

In [45]:
```
reg_prod_agg.mean(numeric_only = True).loc['ABUJA']
```

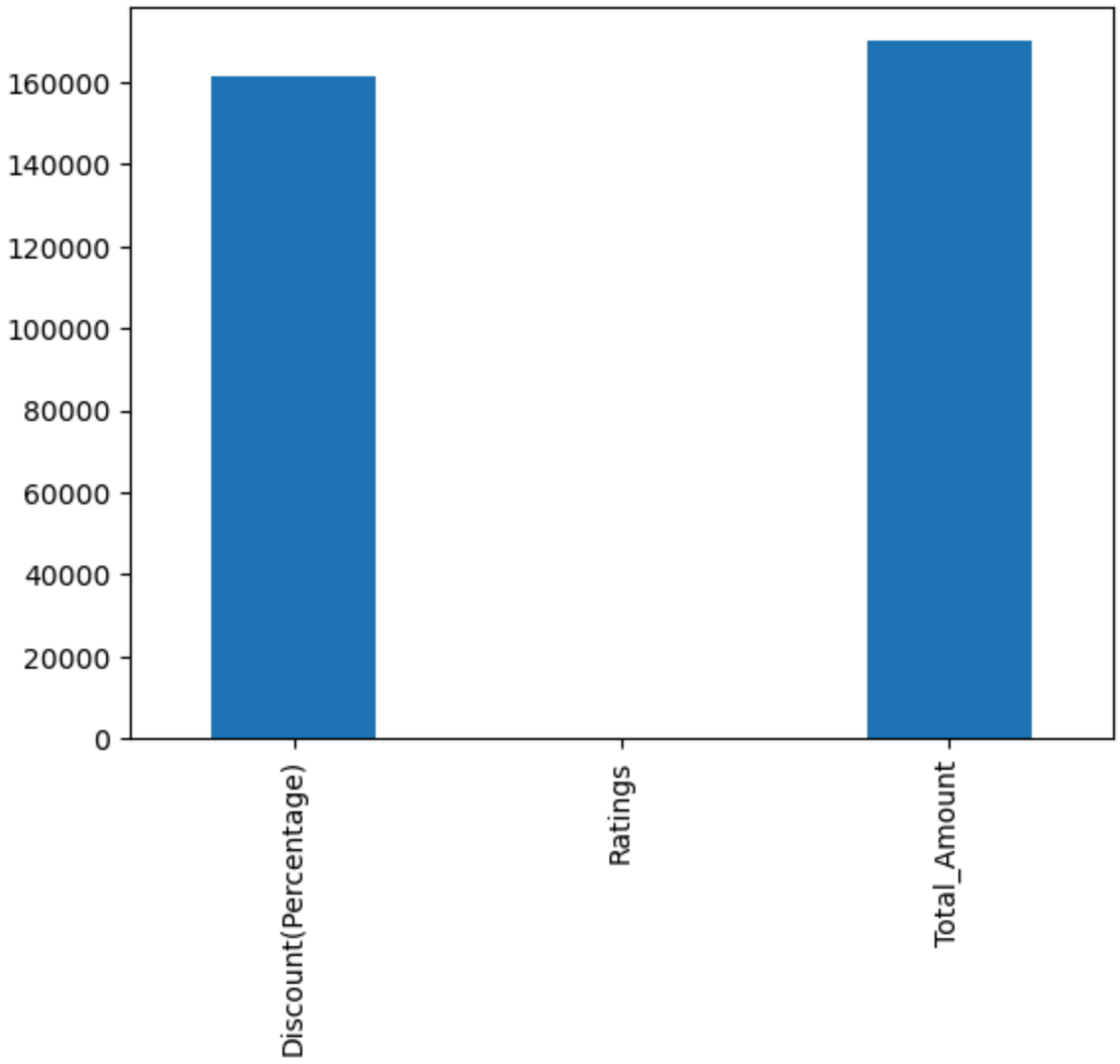
Out[45]:

Product	UnitsSold	UnitPrice	Total_Amount	Discount(Percentage)	Ratings
HEADPHONES	4.750000	2500.75	20002.75	200.0275	5.250000
LAPTOP	6.500000	1.0	6.5	0.065	1.000000
PHONE	2.666667	277977.0	1021786.666667	10217.866667	5.333333
TABLET	2.250000	2500.75	7501.5	75.015	3.000000
UNSPECIFIED	4.000000	5000.5	25001.5	250.015	5.500000

- ***SECTION E : DATA VISUALIZATION.***
- ***DATA VISUALIZATION USING MATPLOTLIB : PROPER CHART SELECTION, LABELING, READABILITY AND CHART INTERPRETATION.***

In [46]:
```
import matplotlib.pyplot as plt

Visual_1 = Product_groupby.agg({'Discount(Percentage)':'max', 'Ratings':'max', 'Tot
Visual_1.plot(kind = 'bar', y = 'Total')
```

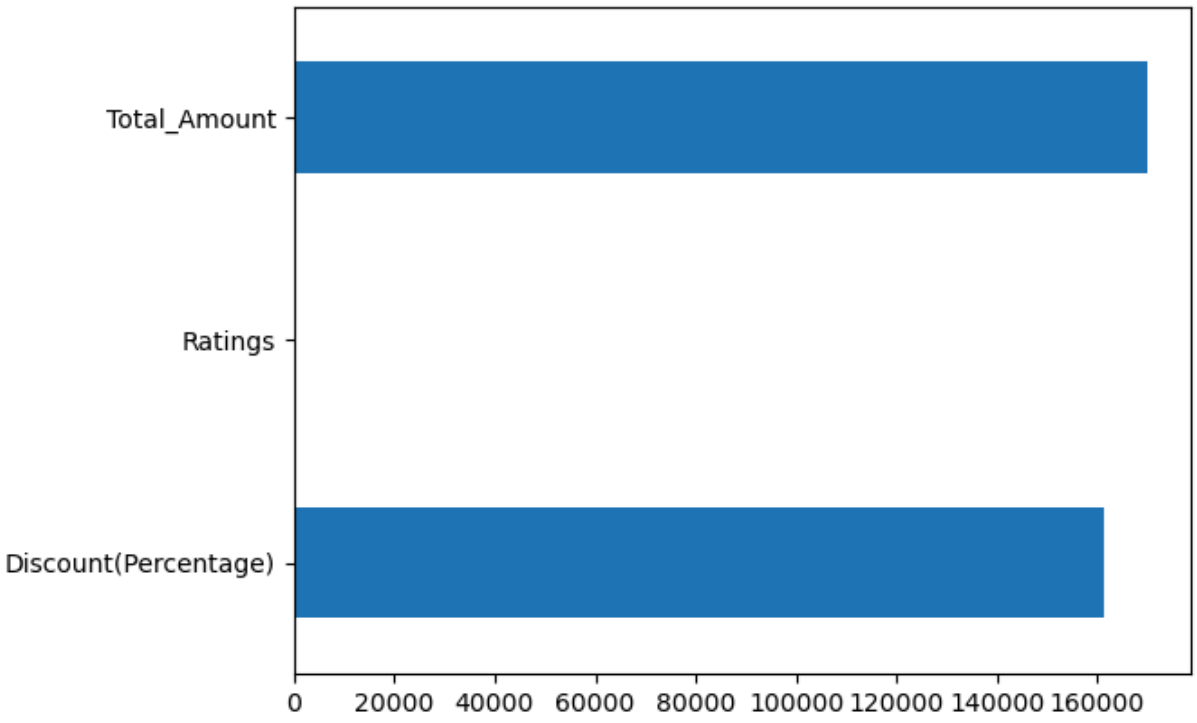
Out[46]:
```
<Axes: >
```



- ***We can also use another type of chart to represent what we just did in the above code. For example, we're going to use the horizontal bar chart to plot.***

In [47]:
```
Visual_1 = Product_groupby.agg({'Discount(Percentage)':'max', 'Ratings':'max', 'Tot
Visual_1.plot(kind = 'barh', y = 'Total')
```

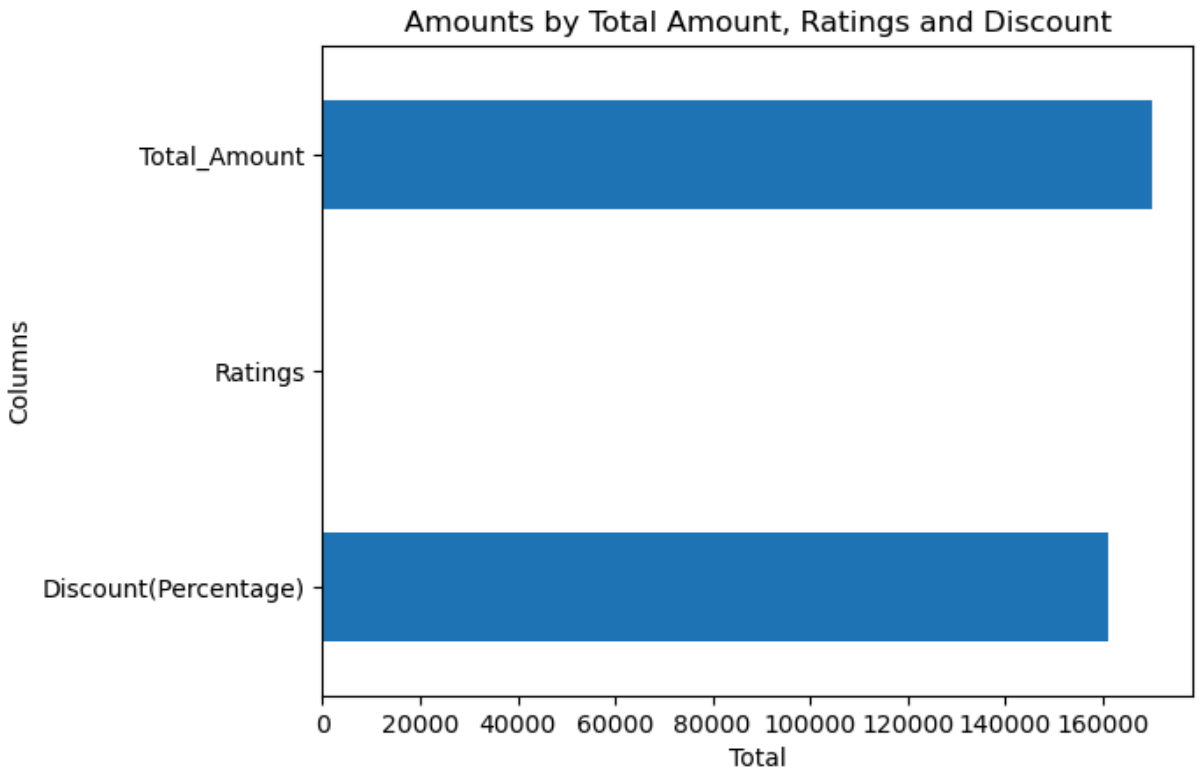
Out[47]: <Axes: >



- ***We can also label our visualized plot in order to be more specific as to what our plot represent.***

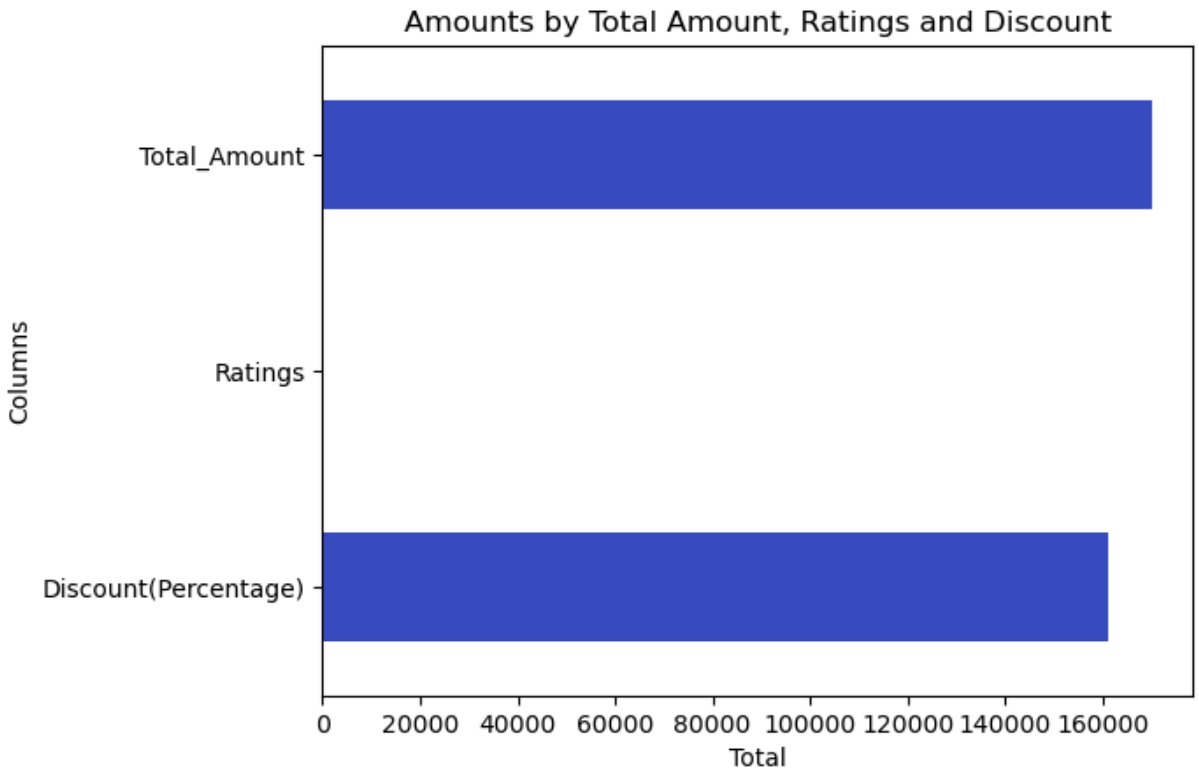
In [48]:
```
Visual_1 = Product_groupby.agg({'Discount(Percentage)':'max', 'Ratings':'max', 'Tot
Visual_1.plot(kind = 'barh', y = 'Total')
plt.xlabel('Total')
plt.ylabel('Columns')
plt.title('Amounts by Total Amount, Ratings and Discount')
```

Out[48]:
```
Text(0.5, 1.0, 'Amounts by Total Amount, Ratings and Discount')
```



- ***Also, we can determine the size of our plot by using the .figure() function.***

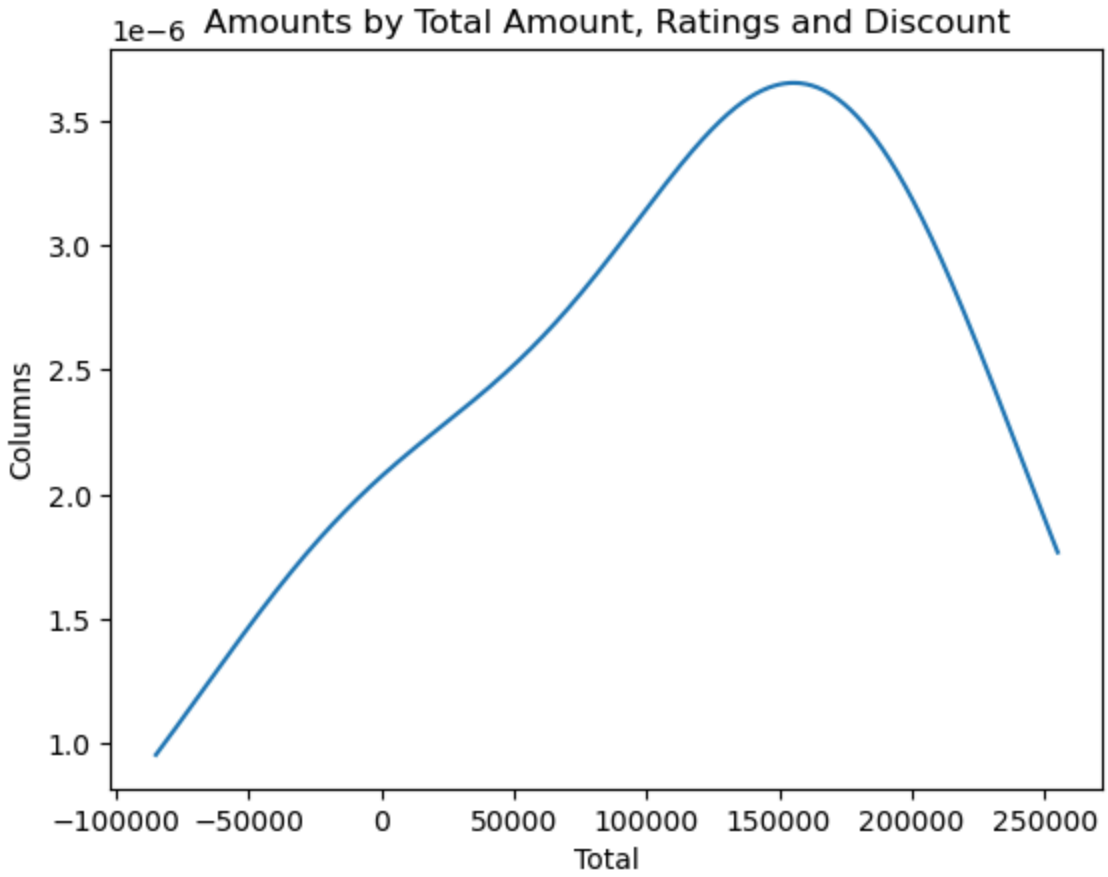
In [49]:
```
Visual_1 = Product_groupby.agg({'Discount(Percentage)':'max', 'Ratings':'max', 'Tot
Visual_1.plot(kind = 'barh', y = 'Total', colormap = 'coolwarm')
plt.xlabel('Total')
plt.ylabel('Columns')
plt.title('Amounts by Total Amount, Ratings and Discount')
plt.figure(figsize = (20,20))
plt.show()
```




```
<Figure size 2000x2000 with 0 Axes>
```

- ***Representing our data with a different plot, e.g the KDE plot.***
- ***The 'KDE(Kernel Density Estimation)' Chart is a smooth, continuous curve that represents the probability distribution of a continuous variable. This is used to show the shape, center and spread of a dataset, it acts as a smoothed-out histogram. Most times, the 'kde' shows negative numbers along the X-axis of the chart.***

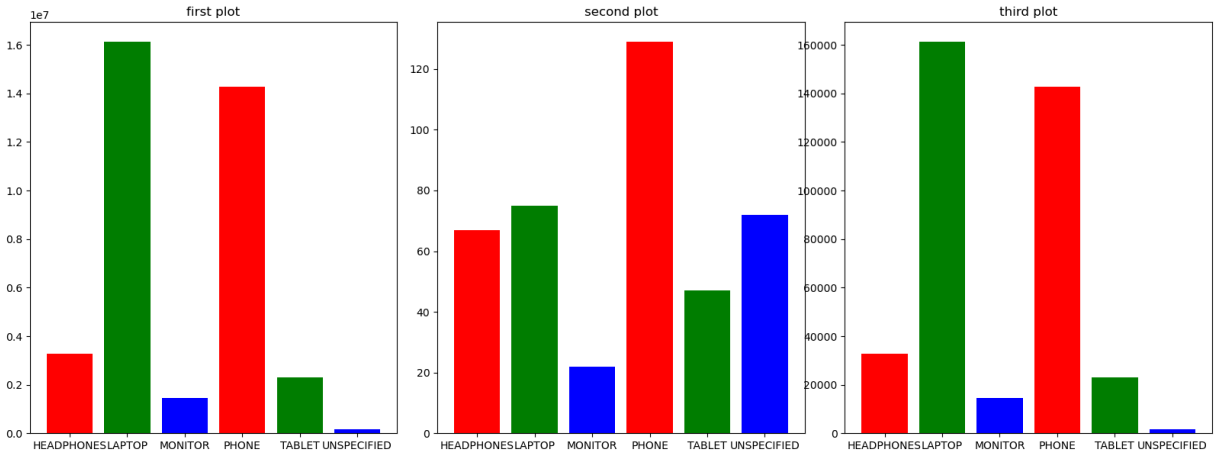
In [50]:
```
Visual_1 = Product_groupby.agg({'Discount(Percentage)':'max', 'Ratings':'max', 'Tot
Visual_1.plot(kind = 'kde', y = 'Total')
plt.xlabel('Total')
plt.ylabel('Columns')
plt.title('Amounts by Total Amount, Ratings and Discount')
plt.figure(figsize = (20,20))
plt.show()
```




```
<Figure size 2000x2000 with 0 Axes>
```

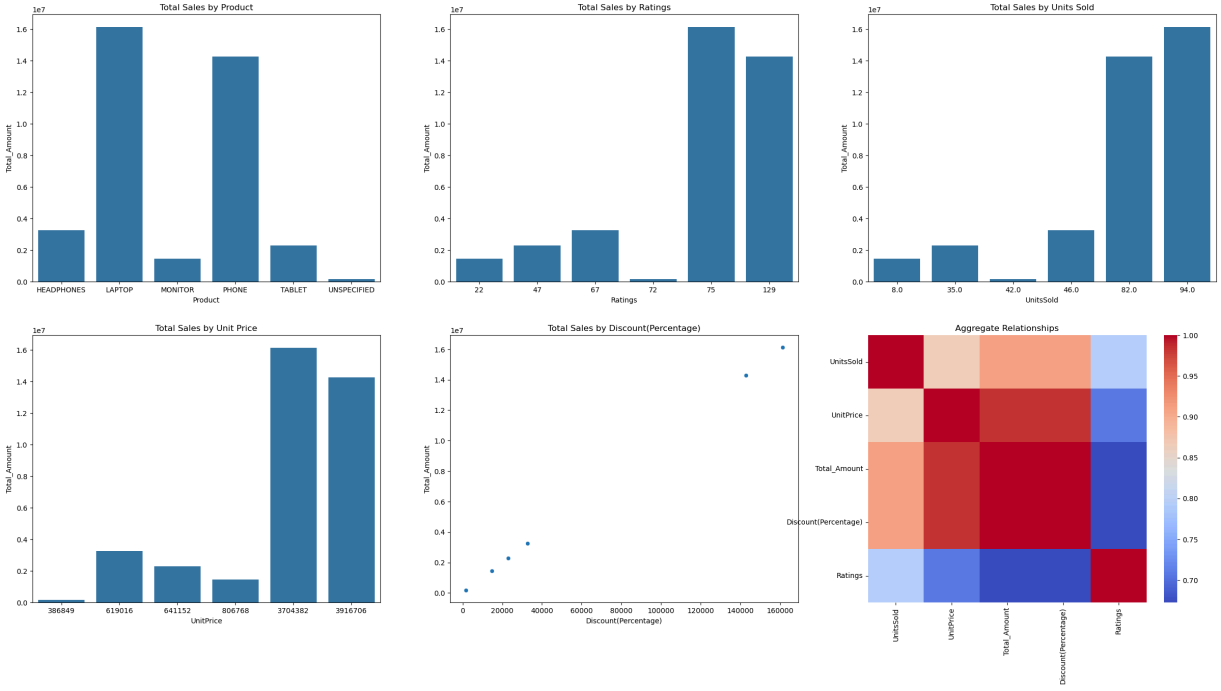
- ***WE CAN ALSO CREATE MULTIPLE SUBPLOTS TO REPRESENT DIFFERENT ASPECTS OF OUR DATA.***

In [51]:
```
plt.figure(figsize=(20,7))

plt.subplot(1,3,1)
plt.bar(Product_groupby.index, Product_groupby['Total_Amount'], color = ['red', 'gr
plt.title('first plot')

plt.subplot(1,3,2)
plt.bar(Product_groupby.index, Product_groupby['Ratings'], color = ['red', 'green',
plt.title('second plot')

plt.subplot(1,3,3)
plt.bar(Product_groupby.index, Product_groupby['Discount(Percentage)'], color = ['r
plt.title('third plot')

plt.subplots_adjust(wspace=0.11, hspace=0.9)

plt.show()
```



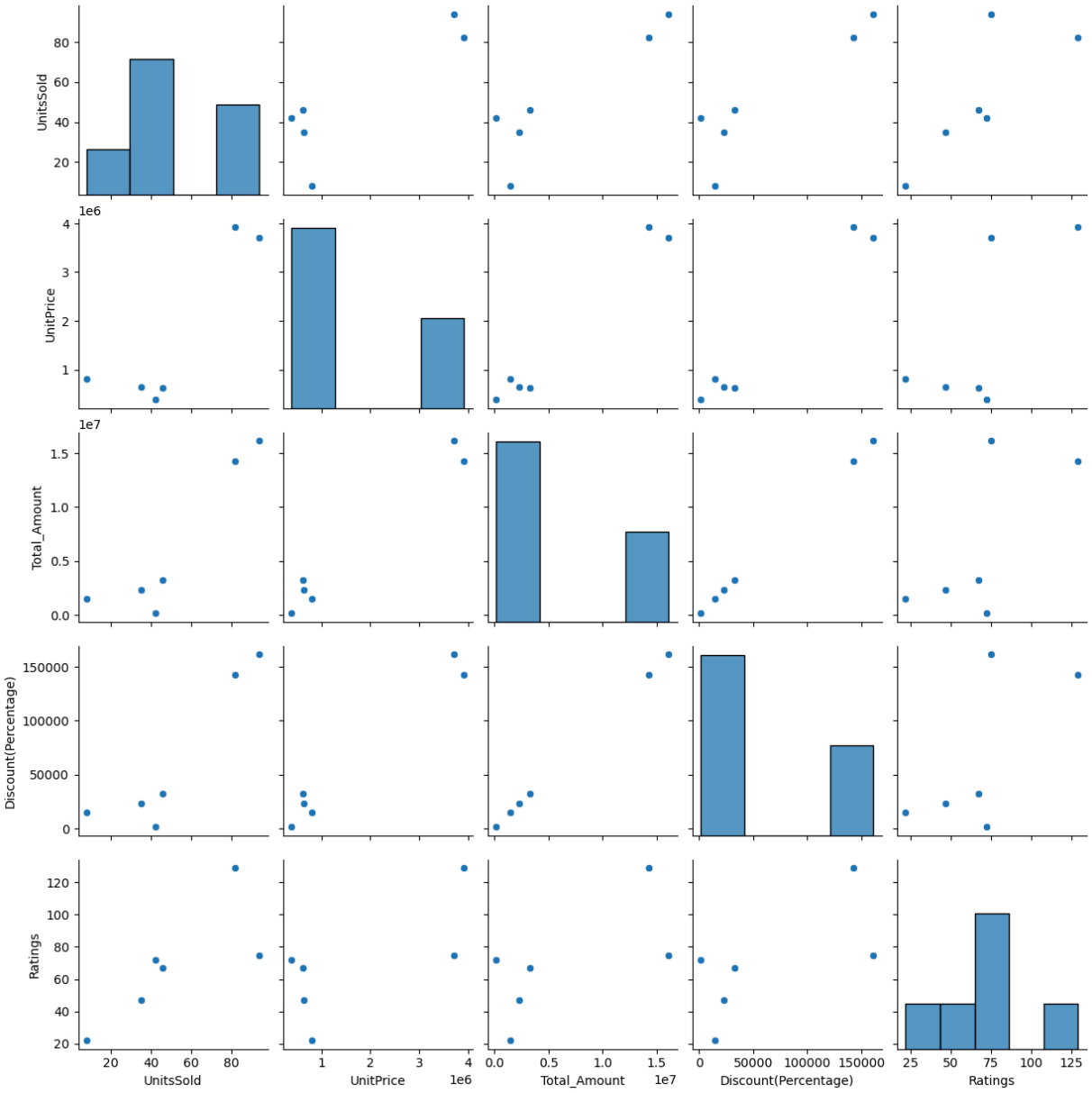
- ***USING SEABORN TO CREATE PLOTS.***

In [52]:
```
import seaborn as sns

fig, axes = plt.subplots(2,3, figsize = (30,15))

sns.barplot(data = Product_groupby, x = 'Product', y = 'Total_Amount', ax=axes[0,0]
axes[0,0].set_title('Total Sales by Product')

sns.barplot(data = Product_groupby, x = 'Ratings', y = 'Total_Amount', ax=axes[0,1]
axes[0,1].set_title('Total Sales by Ratings')

sns.barplot(data = Product_groupby, x = 'UnitsSold', y = 'Total_Amount', ax=axes[0,
axes[0,2].set_title('Total Sales by Units Sold')

sns.barplot(data = Product_groupby, x = 'UnitPrice', y = 'Total_Amount', ax=axes[1,
axes[1,0].set_title('Total Sales by Unit Price')

sns.scatterplot(data = Product_groupby, x = 'Discount(Percentage)', y = 'Total_Amou
axes[1,1].set_title('Total Sales by Discount(Percentage)')

sns.heatmap(Product_groupby.corr(numeric_only=True), ax = axes[1,2], cmap = 'coolwa
axes[1,2].set_title('Aggregate Relationships')

plt.tight_layout
plt.show()
```



In [53]:
```
sns.pairplot(Product_groupby)
```

Out[53]:
```
<seaborn.axisgrid.PairGrid at 0x2c1f82e8d70>
```



KEY INSIGHTS DERIVED AFTER DEEP ANALYSIS

- ***Laptops generated the most revenue. Laptops is the most profitable product, it generated more revenue than any other product in the dataset.***
- ***Phones are the second most profitable.***
- ***Monitors had the lowest revenue among the products. Units sold were relatively low compared to laptops and phones.***
- ***Average customer rating was good. Customer satisfaction is moderately positive. Needs to do better.***
- ***Most popular product are laptops. It's still the most sold product on the dataset.***
- ***Some categories need serious standardization, e.g the "Region" column need to be properly standardized.***
- ***Without proper cleaning, the analysis would have produced misleading results because of the inconsistencies in standardizations. This needs to be fixed.***

- ***Overall data quality was very bad before cleaning. Missing values, duplicates and inconsistent formatting appeared severally across multiple columns and rows. This affects the outcome after cleaning and analysis.***
- ***Laptops and phones are the primary drivers of the business performance. The business should prioritize these products in growth, marketing, varieties and expansion.***

In [ ]: